UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

ACTIVIDAD 2
Problemas y espacios de búsqueda

Datos de entrega	Información
Asignatura	Fundamentos de Inteligencia Artificial
Alumno	Emmanuel Guzmán García
Grupo	172
Docente	Mtro. Javier García Lara
Fecha indicada en el enunciado	18 de octubre de 2026

Repositorio: https://github.com/xbae23/FIA_Parcial2_Emmanuel_Guzman

Demostración: https://xbae23.github.io/FIA_Parcial2_Emmanuel_Guzman/

Portafolio individual versionado en GitHub. Se incluyen planteamiento, procedimiento, evidencia, conclusiones y referencias.

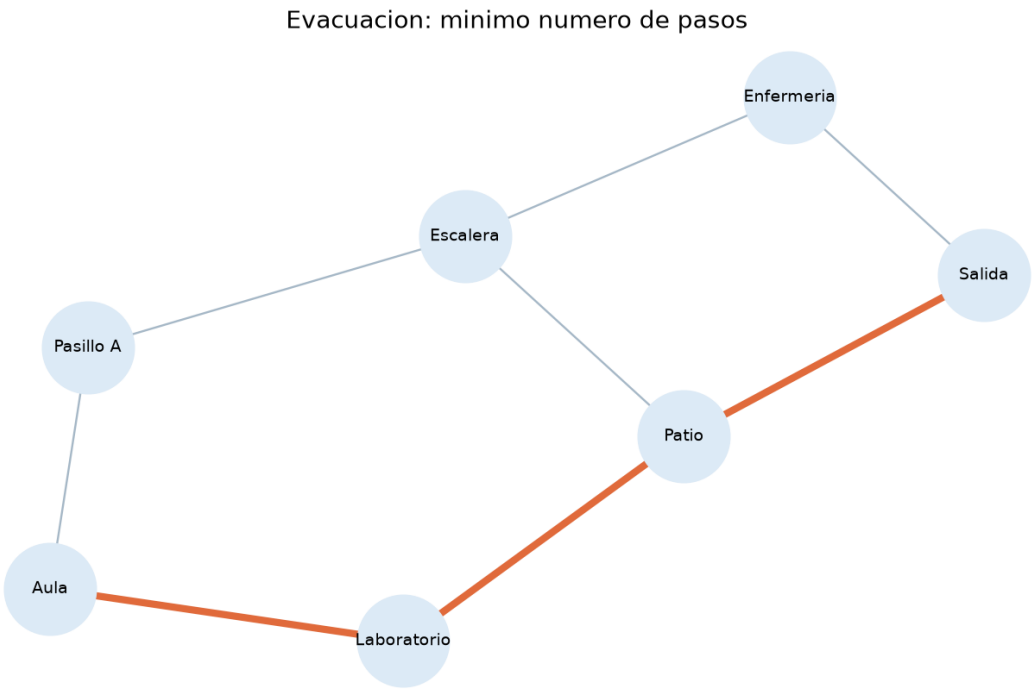
Planteamiento

Se modelan dos situaciones didácticas independientes. Los vértices son estados y las aristas son acciones legales. Los datos del mapa se inventaron con fines pedagógicos y no representan una instalación o ciudad reales.

Problema A: evacuación con mínimo número de pasos

Estado: ubicación actual (aula, pasillo, laboratorio, patio, etc.). **Inicio:** Aula. **Meta:** Salida. **Acción:** pasar a una ubicación adyacente. **Costo:** una acción por arista. **Algoritmo:** búsqueda en anchura (BFS) con cola FIFO y conjunto de visitados. BFS encuentra un camino con mínimo número de aristas en grafos no ponderados.

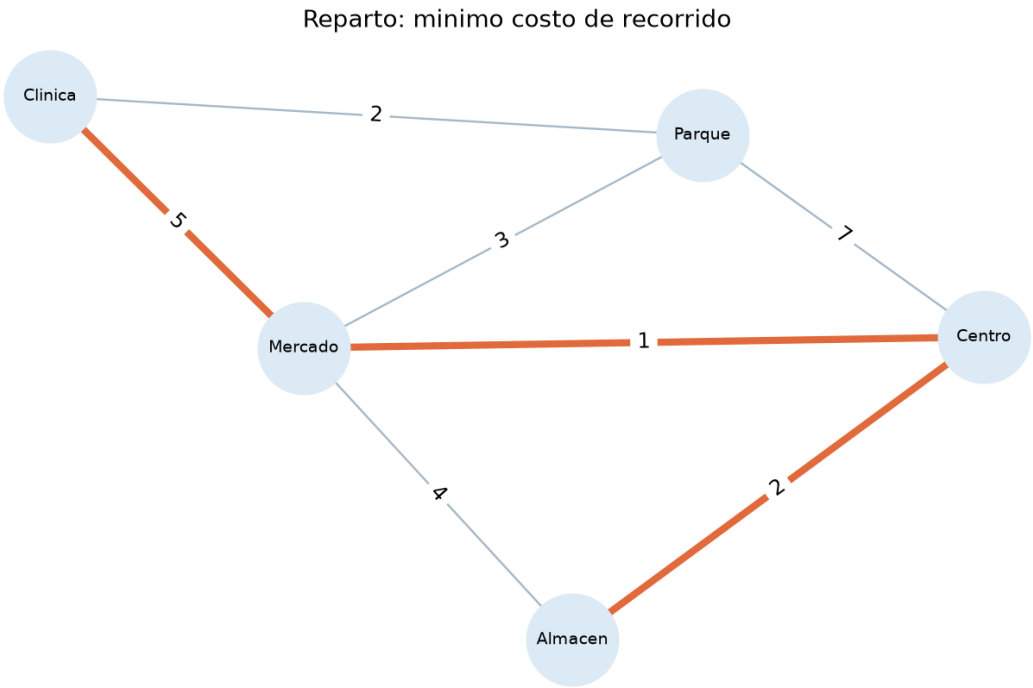
Solución: Aula → Laboratorio → Patio → Salida; 3 pasos, 6 nodos expandidos de 7 estados.



Problema B: reparto con costo de traslado

Estado: punto actual de la red. **Inicio:** Almacen. **Meta:** Clinica. **Acción:** circular por una vía adyacente. **Costo:** valor no negativo de cada arista (unidades de recorrido simuladas). **Algoritmo:** Dijkstra o búsqueda de costo uniforme, con cola de prioridad y relajación de distancias.

Solución: Almacen → Centro → Mercado → Clinica; costo 8 unidades y 5 nodos expandidos.



Procedimiento y complejidad

Problema	Estructura	Tiempo	Memoria
Evacuación / BFS	Cola FIFO + padres	O(V + E)	O(V)

Problema	Estructura	Tiempo	Memoria
Reparto / Dijkstra	Montículo + distancias	O((V + E) log V)	O(V + E)

V es el número de estados y E el de aristas. Los límites consideran listas de adyacencia; el orden alfabético de vecinos usado para reproducibilidad agrega trabajo de ordenación en esta implementación. En reparto hay dos caminos óptimos de costo 8; se muestra el que produce el desempate determinista.

Código y verificación

El código ejecutable y comentado está en **Act2/act2_grafos.py**. Implementa `bfs`, `dijkstra`, `construir_grafos` y exporta diagramas y JSON. La prueba `test_reparto_optimo` comprueba costo y ruta; `test_evacuacion_optima` comprueba tres pasos, inicio y meta. Ejecutar desde la raíz: `python Act2/act2_grafos.py`.

Conclusión

La formulación del costo decide qué camino es mejor: BFS minimiza acciones cuando todas valen lo mismo; Dijkstra minimiza costo acumulado con pesos no negativos. Un modelo equivocado del costo puede producir una ruta matemáticamente óptima pero inútil para la operación real.

Referencias (APA 7.ª ed.)

Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.

Bohm, R. A. (2021). The fundamentals of search algorithms. Nova Science Publishers.

Documentación oficial de NetworkX. (s. f.). Algorithms. https://networkx.org/documentation/stable/reference/algorithms/

Uso de inteligencia artificial y verificación

Se utilizó ChatGPT/Codex para apoyar la organización del reporte, la redacción, el desarrollo de código y gráficos. Se ejecutaron los programas y pruebas locales, y se cotejaron las afirmaciones documentales con las fuentes citadas. Quienes presentan el trabajo deben revisarlo, comprenderlo y poder explicarlo.

Anexo: código Python completo

Archivo ejecutable **Act2/act2_grafos.py**. La numeración facilita la revisión; el archivo del repositorio es la versión directamente ejecutable.

```
001  """Dos problemas de búsqueda reproducibles sobre grafos de NetworkX."""
002  from __future__ import annotations
003
004  import json
005  from collections import deque
006  from pathlib import Path
007
008  import matplotlib
009  matplotlib.use("Agg")
010  import matplotlib.pyplot as plt
011  import networkx as nx
012
013  BASE = Path(__file__).resolve().parent
014  OUT = BASE / "resultados"
015
016
017  def bfs(grafo: nx.Graph, inicio: str, meta: str) -> tuple[list[str], int]:
018      """Busca el camino con menos aristas; cada arista cuesta una acción."""
019      cola = deque([inicio])
020      padres = {inicio: None}
021      expandidos = 0
022      while cola:
023          actual = cola.popleft()
024          expandidos += 1
025          if actual == meta:
026              camino = []
027              while actual is not None:
028                  camino.append(actual)
029                  actual = padres[actual]
030              return list(reversed(camino)), expandidos
031          for vecino in sorted(grafo.neighbors(actual)):
032              if vecino not in padres:
033                  padres[vecino] = actual
034                  cola.append(vecino)
035      return [], expandidos
036
037
038  def dijkstra(grafo: nx.Graph, inicio: str, meta: str) -> tuple[list[str], float, int]:
039      """Uniform-cost search con costos no negativos y desempate determinista."""
040      import heapq
041      cola = [(0.0, inicio)]
042      costo = {inicio: 0.0}
043      padres = {inicio: None}
044      cerrados = set()
045      while cola:
046          distancia, actual = heapq.heappop(cola)
047          if actual in cerrados:
048              continue
049          cerrados.add(actual)
050          if actual == meta:
051              camino = []
052              while actual is not None:
053                  camino.append(actual)
054                  actual = padres[actual]
055              return list(reversed(camino)), distancia, len(cerrados)
056          for vecino in sorted(grafo.neighbors(actual)):
057              peso = float(grafo[actual][vecino]["weight"])
058              if peso < 0:
059                  raise ValueError("Dijkstra requiere costos no negativos")
060              nuevo = distancia + peso
061              if nuevo < costo.get(vecino, float("inf")):
062                  costo[vecino] = nuevo
063                  padres[vecino] = actual
064                  heapq.heappush(cola, (nuevo, vecino))
065      return [], float("inf"), len(cerrados)
066
067
068  def construir_grafos() -> tuple[nx.Graph, nx.Graph]:
069      evacuacion = nx.Graph()
070      evacuacion.add_edges_from([
071          ("Aula", "Pasillo A"), ("Aula", "Laboratorio"),
072          ("Pasillo A", "Escalera"), ("Laboratorio", "Patio"),
073          ("Escalera", "Patio"), ("Escalera", "Enfermeria"),
074          ("Patio", "Salida"), ("Enfermeria", "Salida"),
075      ])
076      reparto = nx.Graph()
077      reparto.add_weighted_edges_from([
078          ("Almacen", "Mercado", 4), ("Almacen", "Centro", 2),
079          ("Mercado", "Clinica", 5), ("Centro", "Mercado", 1),
080          ("Centro", "Parque", 7), ("Mercado", "Parque", 3),
081          ("Parque", "Clinica", 2),
```

```
    ])
    return evacuacion, reparto


def dibujar(grafo: nx.Graph, titulo: str, archivo: Path, camino: list[str]) -> None:
    pos = nx.spring_layout(grafo, seed=11)
    fig, ax = plt.subplots(figsize=(8, 5))
    nx.draw_networkx(grafo, pos, ax=ax, node_size=2000, font_size=8,
                     node_color="#dceaf6", edge_color="#a6b8c7")
    aristas = list(zip(camino, camino[1:]))
    nx.draw_networkx_edges(grafo, pos, edgelist=aristas, ax=ax,
                           width=3.5, edge_color="#e06a3b")
    pesos = nx.get_edge_attributes(grafo, "weight")
    if pesos:
        nx.draw_networkx_edge_labels(grafo, pos, edge_labels=pesos, ax=ax)
    ax.set_title(titulo)
    ax.axis("off")
    fig.tight_layout()
    fig.savefig(archivo, dpi=170)
    plt.close(fig)


def ejecutar() -> dict:
    OUT.mkdir(exist_ok=True)
    evacuacion, reparto = construir_grafos()
    ruta1, n1 = bfs(evacuacion, "Aula", "Salida")
    ruta2, costo2, n2 = dijkstra(reparto, "Almacen", "Clinica")
    dibujar(evacuacion, "Evacuacion: minimo numero de pasos", OUT / "act2_evacuacion.png", ruta1)
    dibujar(reparto, "Reparto: minimo costo de recorrido", OUT / "act2_reparto.png", ruta2)
    datos = {
        "evacuacion": {"inicio": "Aula", "meta": "Salida", "camino": ruta1,
                       "pasos": len(ruta1) - 1, "nodos_expandidos": n1,
                       "estados": evacuacion.number_of_nodes(), "aristas": evacuacion.number_of_edges()},
        "reparto": {"inicio": "Almacen", "meta": "Clinica", "camino": ruta2,
                    "costo": costo2, "nodos_expandidos": n2,
                    "estados": reparto.number_of_nodes(), "aristas": reparto.number_of_edges()},
    }
    (OUT / "act2_resultados.json").write_text(json.dumps(datos, indent=2, ensure_ascii=False), encoding="utf-8")
    print(json.dumps(datos, indent=2, ensure_ascii=False))
    return datos


if __name__ == "__main__":
    ejecutar()
```